

Flask Dashboard, Predict CLI ve Entegrasyon

Ti-6Al-4V Çimentosuz Kalça İmplantı için Surrogate ML Modeli

Ali Edip Mangtay
Yazılım Mühendisliği · 210911044

Mayıs 2026
ENS003 — Digital Twins for Health Sciences · Eğitmen: Prof. Dr. Şenol Pişkin

Bu sunumda

- Surrogate'i son kullanıcının önüne taşıma görevi
- Mimari seçimleri (Flask + Jinja + Plotly + KaTeX)
- LaTeX-paper estetiği ve neden bu yola gidildi
- Sidebar UX: üç sürekli slider + üç preset butonu
- Production'a bakan bug'ları (500 hatası, cache)

Sorumluluğum (yazılım ekibi)

Web app, predict CLI, template/static dosyalar, entegrasyon, `run.sh`.

Server (Python):

- Flask + Jinja2 (HTML render)
- joblib model loading (init'te 10 model)
- Plotly figure JSON üretimi
- / (HTML) ve /predict (JSON) endpoint'leri

Client (browser):

- Plotly.js (CDN, grafik render)
- KaTeX (CDN, denklemler)
- Vanilla JS slider event'leri
- `fetch('/predict?...')` ile AJAX

Tasarım kararı: Bundler yok, framework yok, CDN'den iki kütüphane. Sebep: akademik teslim için **minimum bağımlılık**, jüri makinesinde saniyeler içinde çalışsın.

LaTeX-paper estetiği

Dashboard, bir akademik makale gibi tasarlandı:

- Ortalanmış title-block, yazarlar, instructor, tarih
- *Abstract* bölümü (italik gri)
- Numaralı section'lar (1. Loading conditions, 2. Predicted quantities, ...)
- KaTeX ile inline ve display denklemler
- Booktabs-tarzı tablolar (üst/alt çizgi 1.6pt, orta ince)
- STIX Two Text seri ailesi (akademik PDF görünümü)

Niye bu kadar uğraş?

Demoyu jüriye gösterirken sıradan bir “ML formu” yerine projenin raporunun **interaktif uzantısı** gibi görünmeli. UX hatlarımız Beamer/LaTeX ile uyumlu olmalı.

Üç sürekli slider:

- mass — 40–110 kg
- K — 2.0–6.0 (vücut ağırlığı katsayısı)
- angle — 0° – 20° (kuvvet eğim açısı)

Üç hızlı preset butonu:

- Walking $\Rightarrow K = 2.5$
- Stair climbing $\Rightarrow K = 3.5$
- Running / jumping $\Rightarrow K = 4.5$

Preset butonları sadece K slider'ını set ediyor; mass ve angle korunuyor. Aktivite kategorik girdi değil — K sürekli parametre olduğu için sahte ayrıklık vermemek gerek.

Üç SF gauge:

- Neck
- STEM 1
- STEM 2

Status callout:

min(neck, stem1, stem2)'e göre Safe / Caution / Critical etiketi.

Predict endpoint'i

```
@app.route("/predict")
def predict_endpoint():
    try:
        mass = float(request.args.get("mass", 70.0))
        K = float(request.args.get("k", 2.5))
        angle = float(request.args.get("angle", 0.0))
    except (TypeError, ValueError):
        return jsonify(error="invalid query parameters"), 400
    return jsonify(build_payload(mass, K, angle, models, train_df))
```

build_payload 10 modeli çalıştırıp şunu döndürür:

- input: $(m, K, \theta, F_x, F_z, |F|)$
- metrics: 3 SF + 3 stress + deformation, hepsi $\pm\sigma$
- status: Safe / Caution / Critical
- near: en yakın FEA satırı (normalize (m, K, θ) mesafesi)
- figures: 6 Plotly figure JSON'ı

Bug hikayesi: “Açı slider’ı çalışmıyor”

Şikayet: Slider =0→20 oynatılınca dashboard **donuk**.

Hata zinciri:

- 1 User =5 seçer (grid’de {0,10,20}, off-grid)
- 2 `comparison_chart`’ta filtre `np.isclose(angle, 5) ⇒ 0 satır`
- 3 Boş pandas Series, Plotly figure’da ndarray olarak kalıyor
- 4 Flask jsonify ⇒ **500: ndarray not JSON serializable**
- 5 Frontend if (!resp.ok) return; ⇒ **sessiz fail**
- 6 User eski değerleri görüyor ⇒ “hiçbir şey değişmedi”

Çözüm: `_nearest_value()` ile (K, θ) en yakın grid noktasına snap, legend dinamik etiket, `.tolist()` ile defansif serileştirme.

Snap to nearest slice

```
def _nearest_value(value, options):  
    return float(options[int(np.argmin(np.abs(options - value)))]))  
  
def comparison_chart(rows, ..., K, angle, ...):  
    K_grid = np.sort(rows["K_factor"].unique())  
    angle_grid = np.sort(rows["force_angle_deg"].unique())  
    K_near = _nearest_value(K, K_grid)  
    angle_near = _nearest_value(angle, angle_grid)  
  
    same = rows[  
        np.isclose(rows["K_factor"], K_near)  
        & np.isclose(rows["force_angle_deg"], angle_near)  
    ].sort_values("patient_mass_kg")  
    # ... add Scatter trace with .tolist() casts
```

Legend etiketi otomatik uyum sağlar:

- Tam grid noktası: “FEA training ($K=2.5$, $=0^\circ$)”
- Off-grid: “FEA training (nearest slice $K=2.5$, $=0^\circ$)”

OPERATING POINT

Patient mass

m

70 kg

Body-weight multiplier

K

4.5×BW

Force tilt angle

θ

20°

QUICK PRESETS

Walking
 $K = 2.5$

Stair
Climbing
 $K = 3.5$

Running /
Jumping
 $K = 4.5$

Material

Ti-6Al-4V (annealed)

$\sigma_y = 827$ MPa

$\sigma_u = 918$ MPa

$E = 104.8$ GPa

A Surrogate Model for Stress and Safety Prediction in a Cementless Ti-6Al-4V Hip Implant

Ata Bulut · Recep Kamil Fırat · Ali Edip Mangtay

Department of Computer / Software Engineering, Istinye University
ENS003 — Digital Twins for Health Sciences · Instructor: Şenol Pişkin

May 2026

ABSTRACT

We present a Gaussian-process surrogate model trained on 270 ANSYS Static Structural simulations (162 unique grid points) of a Ti-6Al-4V cementless hip-implant stem. The training grid sweeps six patient masses (50–100 kg), nine body-weight multipliers $K \in \{2, 2.5, \dots, 6\}$, and three force-tilt angles $\theta \in \{0^\circ, 10^\circ, 20^\circ\}$. Given a patient mass, K and θ , the surrogate predicts three minimum safety factors (neck, STEM 1, STEM 2), the maximum von Mises and principal stresses, and the maximum total deformation in milliseconds. 5-fold cross-validation grouped on (m, K, θ) yields $R^2 \approx 0.999$ across all load-bearing targets.

Status — Caution. Marginal: the safety margin is low; experimental verification is recommended.

1. Loading conditions

The femoral force is parameterised by the body-weight multiplier K and the in-plane tilt angle θ :

$$F_x = m g K \cos \theta, \quad F_z = m g K \sin \theta, \quad \|\mathbf{F}\| = m g K.$$

For the operating point selected in the sidebar — $m = 70$ kg, $K = 4.5$, $\theta = 20^\circ$, $F = 3,090$ N — the resulting force is applied to the femoral head.

The surrogate consumes only three independent inputs: patient mass, K , and θ . The force components F_x and F_z shown above are derived analytically from those three — the model never sees activity labels. All ANSYS model constants (mesh, geometry, material, 18-face fixed support) are held fixed across runs and therefore not exposed as inputs.

2. Predicted quantities

Polish detayları

- **Cache disable:** `SEND_FILE_MAX_AGE_DEFAULT=0` $\Rightarrow$ Her CSS/JS düzenlemesi normal refresh'te görünüyor (dev verimliliği)
- **Mobile sidebar:** `@media (max-width: 760px)` ile slide-in panel; hamburger toggle butonu
- **Posterior $\pm\sigma$ annotations:** GPR'nin standart sapması her gauge ve stress bar yanında italik gri olarak gösteriliyor
- **Activity preset butonları:** Sadece K slider'ını set ediyor — mass ve angle korunuyor, user'ın seçimleri yerinde kalıyor
- **Training table scroll + sticky thead:** 270 satır, 460px yükseklik, kaydırırken başlık sabit

CLI + entegrasyon

```
$ ./run.sh predict --mass 70 --k 2.5 --angle 0
{
  "safety_factor_neck_min": 1.553,
  "safety_factor_stem1_min": 1.662,
  "safety_factor_stem2_min": 5.711,
  "max_equivalent_vonmises_stress_Pa": 532000000.0,
  ...
}

$ ./run.sh predict --mass 80 --activity stair_climbing
# K=3.5 preset'e snap'ler, sonra predict
```

Hem dashboard hem CLI **aynı** predict() fonksiyonunu kullanıyor.
preprocess.add_features() ⇒ tek doğruluk kaynağı.

Deploy yolu (açık karar)

Şu an: flask run dev server, Python deps ~ 234 MB

Vercel/Netlify serverless: 250 MB unzipped limit — sınırda, cold start 3-5 sn

Aday yollar:

- 1 Hugging Face Spaces / Render / Railway (Flask olduğu gibi)
- 2 Plotly client-side'a it (~ 40 MB tasarruf)
- 3 **Statik precompute:** (m, K, θ) grid'ini önceden örnekle, ~ 1 MB gzip JSON, frontend trilinear interpolation $\Rightarrow$ GitHub Pages, sıfır backend
- 4 GPR'yi JS'e port et (~ 100 satır), exact predictions

Bu sunumun ardından ekip kararına göre yapılacak.

Teşekkürler.

Sorular?

- `src/app.py`, `src/predict.py`, `src/templates/`, `src/static/`
- `run.sh` (CLI wrapper)
- `scripts/capture_screenshots.py` (dashboard görselleri)
- Sprint logu: `docs/PROJECT_NOTES.md` Sprint 4, 7